

Library Management System API

1. Objective

Develop a **Library Management System REST API** using:

- Node.js
- Express.js
- MySQL
- JWT
- bcryptjs
- Postman

The system must support user authentication, book management, user management, and book borrowing/returning.

2. User Roles

The system has two roles:

Admin

Admin can:

- Add books
- View books
- Update books
- Delete books
- View users
- View all borrowing records

Member

Member can:

- View books
- View their profile
- Borrow books
- Return books
- View their borrowing records

Members must not be allowed to perform admin-only operations.

3. Database

Create a database named:

```
library_management
```

Create the following tables:

```
users
books
borrowings
```

Users Table

Field	Description
id	Primary key
name	User name
email	Unique email
password	Hashed password
role	admin/member
created_at	Registration date

Books Table

Field	Description
id	Primary key
title	Book title
author	Book author
category	Book category
quantity	Total copies
available_quantity	Available copies
created_at	Creation date

Borrowings Table

Field	Description
id	Primary key
user_id	Foreign key to users
book_id	Foreign key to books
borrow_date	Borrow date
return_date	Return date
status	borrowed/returned

Use appropriate **primary keys, foreign keys, constraints, and indexes**.

Use the provided database structure/schema for the implementation.

4. Project Structure

Create the following folder structure:

```
LibraryManagement/
```

```
├── .env
├── package.json
├── server.js
├── schema.sql
├── config/
│   └── db.js
├── controllers/
│   ├── authController.js
│   ├── bookController.js
│   ├── userController.js
│   └── borrowingController.js
├── models/
│   ├── userModel.js
│   ├── bookModel.js
│   └── borrowingModel.js
├── middleware/
│   ├── authMiddleware.js
│   └── roleMiddleware.js
├── routes/
│   ├── authRoutes.js
│   ├── bookRoutes.js
│   ├── userRoutes.js
│   └── borrowingRoutes.js
└── helpers/
    └── hashPassword.js
```

5. Required Packages

Initialize the project:

```
npm init -y
```

Install:

```
npm install express mysql2 jsonwebtoken bcryptjs dotenv
```

Install nodemon:

```
npm install --save-dev nodemon
```

6. Environment Variables

Create a `.env` file containing:

```
PORT=5000
```

```
DB_HOST=localhost
```

```
DB_USER=root
```

```
DB_PASSWORD=
```

```
DB_NAME=library_management
```

```
JWT_SECRET=your_secret_key
```

```
JWT_EXPIRES_IN=1h
```

7. Authentication APIs

Register

POST /api/auth/register

Request:

```
{
  "name": "John",
  "email": "john@gmail.com",
  "password": "123456",
  "role": "member"
}
```

Requirements:

- Validate the input.
- Check whether the email already exists.
- Hash the password using bcryptjs.
- Create the user.
- Allow the role to be `admin` or `member`.
- Default the role to `member` if no role is provided.

Login

POST /api/auth/login

Request:

```
{
  "email": "john@gmail.com",
  "password": "123456"
}
```

Requirements:

- Find the user by email.
- Verify the password.
- Generate a JWT.
- Return the token and user information.

The JWT should contain the user's:

```
id
email
role
```

Current User

GET /api/auth/me

Authentication required.

Return the currently authenticated user's information.

Do not return the password.

8. Middleware

Authentication Middleware

Create:

```
middleware/authMiddleware.js
```

It must:

- Read the Bearer token from the `Authorization` header.
- Verify the JWT.
- Store decoded user information in `req.user`.
- Reject missing, invalid, or expired tokens with 401.

Example:

```
Authorization: Bearer JWT_TOKEN
```

Role Middleware

Create:

```
middleware/roleMiddleware.js
```

Use it to restrict admin-only APIs.

Example:

```
roleMiddleware("admin")
```

Unauthorized roles must receive 403 Forbidden.

9. Book APIs

Implement the following:

```
POST    /api/books
GET     /api/books
GET     /api/books/:id
PUT     /api/books/:id
DELETE  /api/books/:id
```

API	Admin	Member
POST /api/books	✓	X
GET /api/books	✓	✓
GET /api/books/:id	✓	✓
PUT /api/books/:id	✓	X
DELETE /api/books/:id	✓	X

When creating a book:

```
available_quantity = quantity
```

Example:

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "category": "Programming",
  "quantity": 5
}
```

10. User APIs

Admin-only APIs:

```
GET    /api/users
GET    /api/users/:id
PUT    /api/users/:id
DELETE /api/users/:id
```

Requirements:

- Admin can manage users.
- Members cannot access these APIs.
- Password must not be returned in responses.

11. Borrowing APIs

Implement:

```
POST    /api/borrowings
GET     /api/borrowings
GET     /api/borrowings/my
GET     /api/borrowings/:id
PUT     /api/borrowings/:id/return
```

[Borrow Book](#)

```
POST /api/borrowings
```

Authentication required.

Request:

```
{  
  "bookId": 1  
}
```

Before borrowing, check:

- Book exists.
- Book is available.
- User is authenticated.
- User has not already borrowed the same book with status `borrowed`.

On successful borrowing:

```
available_quantity = available_quantity - 1  
status = borrowed
```

Return Book

```
PUT /api/borrowings/:id/return
```

Authentication required.

On return:

```
status = returned  
return_date = current date  
available_quantity = available_quantity + 1
```

A member can return only their own borrowed book.

My Borrowings

```
GET /api/borrowings/my
```

A member can view only their own borrowing records.

All Borrowings

```
GET /api/borrowings
```

Admin only.

Single Borrowing

```
GET /api/borrowings/:id
```

- Admin can view any borrowing record.
- Member can view only their own borrowing record.

12. Business Rules

The following rules must be implemented:

1. A book cannot be borrowed when `available_quantity = 0`.
2. A member cannot borrow the same book while an existing borrowing has `status = borrowed`.
3. A member can return only their own borrowed book.
4. Members cannot perform admin-only operations.

5. Borrowing decreases `available_quantity` by 1.
 6. Returning increases `available_quantity` by 1.
 7. Passwords must always be stored as hashed passwords.
 8. Passwords must not be returned in API responses.
-

13. API List

Authentication

```
POST /api/auth/register
POST /api/auth/login
GET  /api/auth/me
```

Books

```
POST /api/books
GET  /api/books
GET  /api/books/:id
PUT  /api/books/:id
DELETE /api/books/:id
```

Users

```
GET /api/users
GET /api/users/:id
PUT /api/users/:id
DELETE /api/users/:id
```

Borrowings

```
POST /api/borrowings
GET  /api/borrowings
GET  /api/borrowings/my
GET  /api/borrowings/:id
PUT  /api/borrowings/:id/return
```

14. Postman Testing

Test the following:

Authentication

- Register admin and member users.
- Login as admin and member.
- Test invalid login.
- Test API without a token.
- Test invalid/expired token.
- Test `/api/auth/me`.

Books

- Admin creates a book.
- Member attempts to create a book.
- Admin and member view books.
- Admin updates a book.
- Member attempts to update a book.
- Admin deletes a book.
- Member attempts to delete a book.

- Admin views users.
- Member attempts to view users.
- Admin views a single user.
- Admin updates a user.
- Member attempts to update a user.
- Admin deletes a user.
- Member attempts to delete a user.

Borrowings

- Member borrows a book.
- Member views their borrowings.
- Member returns a book.
- Member attempts to borrow an unavailable book.
- Member attempts to borrow the same book twice.
- Member attempts to return another member's book.
- Admin views all borrowing records.

Expected Result

The completed API should demonstrate:

- Node.js and Express.js
- REST API development
- MySQL database operations
- CRUD operations
- Password hashing
- JWT authentication
- Authentication middleware
- Role-Based Access Control
- Book management
- User management
- Borrowing and returning books
- Postman API testing